Rapport de Mi-Étape
Projet Puissance 4 3D

Maher Moujahed

Moussa Chakroun

Youssef Rais

Mohamed Fadhel Bouzidi

2 Mai 2026

1 Introduction

Dans le cadre de notre projet de Génie Logiciel, notre équipe développe une modélisation en C++ d'un jeu de Puissance 4 en 3D. Le but est d'aligner 4 perles dans un espace tridimensionnel (plateau de 20 bâtonnets 4x5x4). Ce document présente notre organisation, nos premiers défis techniques d'implémentation, ainsi que l'état de notre version de test actuelle.

2 Présentation de l'Équipe et Répartition des Rôles

Notre équipe de 5 membres s'est répartie les responsabilités de management suivantes :

- **Youssef Rais** : Responsable de la gestion de projet (répartition de l'écriture du code)
- **Moussa Chakroun** : Responsable de l'élaboration des tests unitaires et du respect du principe d'encapsulation.
- **Maher Moujahed** : Responsable de la documentation du code et de la charte d'écriture / co-résponsable de la rédaction des rapports laTeX
- **Mohamed Fadhel Bouzidi** : Responsable de la gestion des commits sur notre forge Git et co-rédacteur de ce rapport LaTeX.

3 Modélisation et Défis Techniques Rencontrés

Lors du démarrage du projet, la conceptualisation de la 3D a posé plusieurs défis de modélisation.

3.1 Définition de l'espace de jeu (Le plateau 3D)

Le premier problème rencontré a été de trouver la structure de données la plus adaptée pour représenter le plateau. Nous avons opté pour un tableau multidimensionnel en 3D encapsulé dans la classe `Board`. Cela nous permet d'isoler chaque coordonnée facilement.

```
class Board {
private:
    // grille[rangee][colonne][niveau]
    Cell grille[4][5][4];

public:
    Board() { /* Initialisation de toutes les cases a VIDE */ }
};
```

3.2 Conception de la Gravité et des Niveaux

Le second défi technique majeur fut la simulation de la "gravité". Le joueur ne choisissant que le bâtonnet, la perle doit "tomber" au niveau le plus bas disponible. Nous avons implémenté cette mécanique en parcourant les niveaux de bas en haut :

```
void placerPerle(Move coup, Couleur coul) {
    for (int n = 0; n < 4; n++) { // Parcours de bas en haut
        if (!grille[coup.rangee][coup.colonne][n].occupee) {
            grille[coup.rangee][coup.colonne][n].occupee = true;
            grille[coup.rangee][coup.colonne][n].couleur = coul;
            return; // La perle s'arrete au premier niveau vide
        }
    }
}
```

4 Algorithmique : La Détection de Victoire (`WinChecker`)

Un défi intellectuel majeur a été de concevoir l'algorithme permettant de vérifier si un coup est gagnant dans un environnement 3D. Contrairement à un plateau 2D classique (où l'on vérifie 4 directions), l'espace tridimensionnel implique de vérifier **13 directions distinctes**.

L'idée derrière notre algorithme repose sur l'exploration vectorielle autour du dernier coup joué :

1. **Le point d'origine :** Plutôt que de reparcourir l'intégralité du plateau à chaque tour (ce qui serait coûteux en temps de calcul), l'algorithme part de la position exacte (rangée, colonne, niveau) de la dernière perle placée.
2. **Exploration bidirectionnelle :** Pour chacune des 13 directions de l'espace 3D, l'algorithme "marche" dans un sens (ex : vers la gauche) puis dans le sens opposé (ex : vers la droite), en comptant les perles consécutives de la même couleur.
3. **Condition d'arrêt :** Si le compteur atteint 4, la partie est gagnée. Si l'exploration rencontre une case vide, une perle adverse, ou les bords du plateau, elle s'arrête et passe à la direction suivante.

Cette approche garantit une complexité temporelle minimale, essentielle pour l'optimisation future de notre Intelligence Artificielle.

5 État d'Avancement Général

À ce stade du projet, **aucune classe n'est considérée comme totalement terminée**. Tout est encore en cours de construction et d'ajustement. Notre focus actuel est de faire fonctionner les classes ensemble de manière cohérente.

— **Infrastructure et Intégration :** Les classes `Board`, `Player`,`Game`, `Cell`, `Move`, et l'énumération `Couleurs` sont presque abouties. Elles représentent l'infrastructure vitale du jeu et parviennent désormais à communiquer entre elles. Nous travaillons actuellement sur l'amélioration de l'affichage terminal pour rendre le plateau 3D lisible par l'utilisateur humain.
— **Compilation et Version de Test :** Bien que notre projet ne se compile pas encore intégralement en dehors de tout IDE dans sa version finale, nous avons réussi à concevoir un `Makefile` fonctionnel pour une version restreinte.
— **Éléments en attente :** Cette version de test nous sert uniquement à vérifier que l'on peut jouer à tour de rôle et que le plateau stocke correctement l'information. L'intégration complète de la classe `WinChecker` et des stratégies de l'`AIPlayer` fera l'objet de notre prochain sprint de développement.

6 Conclusion

Notre priorité pour la suite immédiate du projet est de consolider cette version de base en y intégrant la détection de victoire 3D complète. Une fois cette étape technique validée, nous pourrons sereinement démarrer la documentation Doxygen (déja incorporée partiellement avec chaque classe) et nous concentrer sur l'élaboration de notre Intelligence Artificielle et d'optimiser la stratégie gagnate en vue du tournoi.